

Practical No. 2

Aim:- To understand & implement Views & Indexing in MySQL for performance optimization in a student Course Management system, enabling faster queries & simplified reporting.

Theory :-

(1) Views :-

- A view is a virtual table based on the result of a SELECT query.
- Advantages:- Simplifies complex queries, enhances security, allows limited updates/deletes.
- Update/Delete is allowed only if the view is based on a single table without aggregation.

(2) Indexing :-

- An index improves search & query performance.
- Types :- Primary key, Unique, Normal, Composite.
- Advantages:- faster SELECT query better JOIN performance.
- Trade-off:- slightly slower INSERT/UPDATE/DELETE ; extra storage usage.

(3) Optimization :-

- Views make reporting easy without repeatedly writing complex queries.
- Indexes reduce query execution time on large tables.
- Combined, they enhance database efficiency.

```
CREATE DATABASE IF NOT EXISTS StudentDB;
USE StudentDB;

CREATE TABLE IF NOT EXISTS Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT,
    department VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL,
    credits INT
);

-- Enrollment Table (Composite Primary Key)
CREATE TABLE IF NOT EXISTS Enrollment (
    student_id INT,
    course_id INT,
    grade CHAR(1),
    PRIMARY KEY (student_id, course_id);
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
```

```
);
```

```
INSERT INTO Students VALUES
```

```
(1,'Alice',20,'CS'),
```

```
(2,'Bob',21,'EE'),
```

```
(3,'Charlie',22,'ME'),
```

```
(4,'David',23,'CS');
```

```
INSERT INTO Courses VALUES
```

```
(101,'DBMS',4),
```

```
(102,'OS',3),
```

```
(103,'Networks',3),
```

```
(104,'AI',4);
```

```
INSERT INTO Enrollment VALUES
```

```
(1,101,'A'),
```

```
(1,102,'B'),
```

```
(2,101,'B'),
```

```
(3,103,'A'),
```

```
(4,101,'B'),
```

```
(4,104,'A');
```

```
-- Students older than 21
```

```
SELECT * FROM Students
```

```
WHERE age > 21;
```

```
-- Students from CS department
```

```
SELECT * FROM Students
```

```
WHERE department = 'CS';
```

```
-- Students in alphabetical order
```

```

SELECT * FROM Students
ORDER BY name ASC;

-- Courses sorted by credits (descending)
SELECT * FROM Courses
ORDER BY credits DESC;

-- Count number of students per course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollment e ON c.course_id = e.course_id
GROUP BY c.course_name;

-- Average age of students per department
SELECT department, AVG(age) AS avg_age
FROM Students
GROUP BY department;

-- Students enrolled in courses with credits > 3
SELECT name
FROM Students
WHERE student_id IN (
    SELECT student_id
    FROM Enrollment
    WHERE course_id IN (
        SELECT course_id
        FROM Courses
        WHERE credits > 3
    )
)

```

);

-- Courses having more than 1 student

SELECT course_name

FROM Courses

WHERE course_id IN (

 SELECT course_id

 FROM Enrollment

 GROUP BY course_id

 HAVING COUNT(student_id) > 1

);

CREATE INDEX idx_student_age ON Students(age);

CREATE INDEX idx_course_credits ON Courses(credits);

CREATE INDEX idx_enrollment ON Enrollment(student_id, course_id);

EXPLAIN SELECT * FROM Students WHERE age > 21;

EXPLAIN SELECT * FROM Courses WHERE credits > 3;

SELECT * FROM Students;

SELECT * FROM Courses;

SELECT * FROM Enrollment;

Result Grid		Filter Rows:		Export:	Wrap Cell Content: <u>IA</u>
	student_id	name	department	course_name	grade
▶	1	Alice	CS	DBMS	A+
	1	Alice	CS	OS	B
	2	Bob	EE	DBMS	B
	3	Charlie	ME	Networks	A

#

Conclusion

- Views simplify reporting Δ allow limited updates/deletes on virtual tables.
- Indexes significantly improve query performance, especially on frequently searched columns.
- Combining views Δ indexes optimizes the student Course Management System for faster data retrieval.
- Lab successfully demonstrates performance improvement Δ easier data management using views & indexes.